

Quick Revision — Stream Processing & Analytics

Last-night sheet: numbers table · mnemonics · 80 rapid-fire Q&As · conflicts at a glance · Model-QP-only topics. Unit 4 (Kafka intro) has its own notes; only a few Unit 4 one-liners are repeated here because Part A covers the whole syllabus.

Paper names: **SEE Feb'26** = *SPA SEE 2022.pdf* (the exam it contains is dated Feb 2026) · **Model QP** · **CIE-1 Dec'24** and **CIE-2 Jan'25** (files in *QP/CIE/2021*, papers dated 26/12/2024 & 27/01/2025) · **CIE-1 Nov'25**, **CIE-2 Dec'25**, **CIE-3 Jan'26** (files in *QP/CIE/2022*). Blue **highlight** = must-remember. **BOOK p.x ↗** = Streaming-Data[1].pdf printed page (PDF = +20) or Kafka.pdf PDF page.

1 · Exam pattern in one box

Part A (20)	Q1: 10 questions × 2 marks, whole syllabus (mix of "define/list/draw", fill-ups and one-line scenario questions).
Part B (80)	5 questions × 16: Q2 compulsory (Unit 1) ; Q3/4 (Unit 2), Q5/6 (Unit 3), Q7/8 (Unit 4), Q9/10 (Unit 5) — choose one in each; max 3 sub-parts; SEE Feb'26 splits like 8+8, 10+6, 6+10, 8+8.
Marks tip	10-mark ≈ 1 diagram + 6–8 points + example; 6-mark ≈ diagram + 4 points; 2-mark ≈ 2 lines.

2 · Numbers & facts table

Fact	Value	Where
Latency: hard / soft / near	μs –ms / ms–s / s–min	Unit 1, Table 1.1 BOOK p.5 ↗
Interaction patterns	5 (req/resp, req/ack, pub/sub, one-way, stream)	Unit 1 BOOK p.15 ↗
Streaming blueprint tiers	5 + optional long-term storage	Unit 1 BOOK p.9 ↗
Google search stream example	46,000 events/s	Unit 1 BOOK p.27 ↗
NFL RFID tag rate	25 samples/s, 20 receivers, analysed within 0.5 s	Unit 1 BOOK p.23 ↗
HML overhead reduction vs RBML	43%	Unit 1 BOOK p.37 ↗
Failure points: queue path / stream processing	6 (Fig 3.12) / 7 (Fig 4.15)	Unit 2 BOOK p.48 ↗ BOOK p.73 ↗
State-machine replication cost	$k + 1$ times resources	Unit 2 BOOK p.75 ↗
Delivery semantics	3 (at-most / at-least / exactly-once)	Unit 2 BOOK p.47 ↗
Spark Streaming batch interval	< 0.5 s	Unit 2 BOOK p.66 ↗
Gas turbine data	~1 TB/hour	Unit 2 BOOK p.63 ↗
Fitbit window	length 30 min, slide 5 s	Unit 3 BOOK p.82 ↗
Reservoir sampling probability	k/n (16th item, $k=15 \Rightarrow 15/16$)	Unit 3 BOOK p.87 ↗
HyperLogLog: 1 billion items	2% error in 1.5 KB; precision 6 bits $\Rightarrow$ 64 registers	Unit 3 BOOK p.90 ↗
Count-Min: 8×128 array	$\approx 1.5\%$ error with 99.6% probability	Unit 3 BOOK p.92 ↗
Bloom filter space	$m/8$ bytes	Unit 3 BOOK p.93 ↗
Memory speeds	SSD seek 100,000 ns; DRAM 100 ns; L1 0.5 ns	Unit 3 BOOK p.100 ↗
Three long-term write options	message-by-message / batch from analysis / via queue + batch loader	Unit 3 BOOK p.97 ↗
Kafka Connect added in	Kafka 0.9	Unit 5 BOOK p.327 ↗
Kafka Streams introduced	0.10.0	Unit 5 BOOK p.596 ↗
Connect REST port	8083	Unit 5 BOOK p.342 ↗
Stream-table external lookup cost	5–15 ms per lookup; DB ~10K ev/s vs 100K–500K ev/s	Unit 5 BOOK p.619 ↗
Stock example window	5 s, advance 1 s	Unit 5 BOOK p.633 ↗
Kafka Streams tasks	= number of input partitions	Unit 5 BOOK p.644 ↗

3 · Mnemonics (all in one place)

C-M-A-I-D(+L)	Collection → Message queue → Analysis → In-memory store → Data access (+ Long-term storage)
R-R-A-P-O-S	Request/Response, Request/Acknowledge, Pub/sub, One-way, Stream
RBML = front door, SBML = back door	Receiver logs on receipt; sender logs before sending; HML = both, async
μs, ms, minutes	Hard (none), Soft (low), Near (high tolerance)
"CCUNI"	Chronological, Continuous, Unique, Non-homogeneous, Imperfect (streaming data)
I-N-P-C-O-M-D	7 failure points: Incoming stream, Network, Processor, Connection-to-output, Output, Manager, Driver
S-S-F-S	Spark = small batches; Storm = single tuple; Flink = stream-first; Samza = stage-wise (YARN + Kafka)
"One Cat Really Dislikes"	One-pass, Concept drift, Resource limits, Domain constraints
Reservoir · HLL · Count-Min · Bloom	Sampling · Distinct · Frequency · Membership
RRW-WWW	Read-through, Refresh-ahead Write-through, Write-around, Write-back
T-R-H-D-T-S-F-C	Pipeline: Timeliness, Reliability, High throughput, Data formats, Transformations, Security, Failure handling, Coupling
Trigger = When, Eviction = Out	Trigger fires processing; eviction removes data
Sliding overlaps, Tumbling drains	Sliding: length + interval; Tumbling: window full ⇒ empty

4 · Conflicts & traps at a glance

⚠ ONE-GLANCE TRAP LIST (details in each unit)

1. **Soft vs near examples** — book puts stock quotes under soft, slides put stock trading under near. Quote the Table 1.1 latencies, use slide examples (Unit 1 §1.1).
2. **Checkpointing** — book: bad fit for streaming; CIE scheme: checkpoint + logging together. Write both (Unit 1 §6).
3. **Which logging?** power-failure at the node ⇒ RBML; crash of the receiver with senders able to resend ⇒ SBML (Unit 1).
4. **6 vs 7 failure points** — queue path 6, stream processing 7 (Unit 2).
5. **Exactly-once in Kafka** — 2017 book says not provided; 2021 Kafka book says yes via idempotence + transactions (Unit 2).
6. **Fig 4.11 caption** says at-least-once but shows at-most-once (Unit 2).
7. **Sliding vs hopping** — same thing in different books (Unit 3/5).
8. **Table 5.3 / text** — Flink "only" native tumbling support, Storm can implement easily (Unit 3).
9. **Stream time/event time** (Streaming Data) vs event/log-append/processing time (Kafka) — don't mix (Unit 3/5).
10. **Range-assignor question (Unit 4, CIE-1 Nov'25 & SEE Feb'26 8a)** — official solution gives C1:p0-2, C2:p3-5; with C3: 2 each; the "issue" it states (uneven load) does not really arise with 6 partitions/3 consumers; the true issue is a consumer beyond the partition count sits idle (e.g. 7th consumer) and Range can be unbalanced across several topics. See Unit 4 notes; write the official assignment + mention these.
11. **Paper file names** — folder "2021" holds Dec'24/Jan'25 papers; folder "2022" holds Nov'25–Feb'26 papers.

5 · 80 rapid-fire Q&As (likely MCQs / Part A)

Unit 1 (1–20)

1. Hard real-time example?	Pacemaker / anti-lock brakes.
2. Which real-time class has seconds–minutes latency?	Near.
3. Streaming data system is ___ real-time.	Non-hard.
4. Name the five tiers of the blueprint.	Collection, message queuing, analysis, in-memory store, data access.
5. Optional tier in the blueprint?	Long-term storage.
6. Security is added how?	Overlaid: authentication + authorization at tiers.
7. Vertical scaling means?	Add resources to the same server.
8. Horizontal scaling means?	Add more servers.
9. Scaling goal for streaming tiers?	Horizontal.
10. Pattern with acknowledgement token?	Request/acknowledge.
11. Fire-and-forget pattern?	One-way.
12. Pattern where the service becomes the client?	Stream.
13. Half-async means?	Only one side (client or service) is asynchronous.
14. Clickstream collection uses which pattern?	Stream.
15. Why are request/response services easy to scale?	Stateless: any instance can serve any client.
16. Rule for buffering layer in stream scaling?	No business logic on messages read from the stream.
17. Checkpointing stores?	Global snapshot of whole system state.
18. Data node loses messages after power failure — technique?	RBML.
19. SBML deletes a logged message when?	After the next tier acknowledges it.
20. HML advantage?	Same safety as RBML+SBML with lower overhead (async, one storage).

Unit 2 (21–40)

21. Three components of message queuing?	Producer, broker, consumer.
22. Main purpose of queue tier?	Decouple collection from analysis tier.
23. Problem if consumers are slower than producers without a queue?	Back-pressure $\Rightarrow$ data lost.
24. Durable messaging supports?	Offline consumers, replay, disaster recovery.
25. At-most-once means?	May be lost, never re-read.
26. At-least-once means?	Never lost, may be duplicated.
27. Exactly-once workaround techniques?	No blind retry; store last-message metadata.
28. Kafka metadata to store for exactly-once?	Offset.
29. Two data-quality bonuses?	Message auditing, duplicate detection.
30. Broker crash mitigation?	Ack after disk write, replication, little in memory.
31. In-flight data?	Data in motion, never at rest.
32. Continuous query model?	Query registered once; data flows through it; results pushed.
33. DBMS query model?	One-time pull on static data.
34. Storm master node?	Nimbus (supervisors are workers).
35. Flink master/worker?	JobManager / TaskManager.
36. Samza depends on?	YARN and Kafka.
37. Tuple-at-a-time vs stage-wise?	Storm vs Samza.
38. Idempotent job needed for?	At-least-once delivery.
39. Streaming manager fails — what happens?	Processors run headless.
40. Rollback recovery uses?	Checkpoint + log replay (lower overhead, slower recovery).

Unit 3 (41–58)

41. Stream time?	Time event enters the system.
42. Event time?	Time event occurred at source.
43. Difference between them called?	Time skew.
44. Trigger policy?	Rule for when to process the window.
45. Eviction policy?	Rule for removing data from the window.
46. Sliding window parameters?	Window length (eviction) + sliding interval (trigger).
47. Tumbling window overlap?	None.
48. Two tumbling types?	Count-based, temporal.
49. Spark Streaming time support?	Stream time only.
50. Concept drift?	Statistical properties of data change over time.
51. Load shedding?	Dropping tuples that can't be processed in time.
52. Reservoir sampling inclusion probability?	k/n .
53. Algorithm for distinct counts?	HyperLogLog.
54. Algorithm for frequency?	Count-Min Sketch.
55. Bloom filter errors?	False positives yes, false negatives no.
56. Batch loader?	Reads batches from queue tier, writes to long-term store.
57. Cache that acknowledges before store write?	Write-back.
58. IMDB design philosophy?	Memory-first, disk for durability.

Unit 4 (59–66) — repeated from past papers; full notes in Unit 4 PDF

59. Three ways of sending messages with a Kafka producer?	Fire-and-forget, synchronous, asynchronous (callback).
60. At-least-once producer settings (SEE Feb'26 1.9)?	acks = all, retries > 0 (large), enable.idempotence = false (true adds duplicate protection) — check Unit 4 notes for the exact wording.
61. Role of MirrorMaker?	Copies (mirrors) data between Kafka clusters. BOOK p.369 ↗
62. Consumer group purpose?	Share partitions among consumers for scaling; one partition → one consumer in a group.
63. Eager vs cooperative rebalance?	Stop-all-and-reassign vs move only affected partitions (CIE-1 Nov'25 A5).
64. Range assignor, 6 partitions, 2 consumers?	C1: P0–P2, C2: P3–P5.
65. Why Kafka over other pub/sub systems?	Durable log, scalable, replicated, high throughput, ecosystem (Connect/Streams).
66. Kafka ecosystem inputs/outputs?	Applications create data → Kafka → metrics, reports, loops back.

Unit 5 (67–80)

67. Kafka's main value in a pipeline?	Reliable buffer decoupling producers/consumers.
68. ETL?	Transform inside the pipeline.
69. ELT?	Load raw, transform at target.
70. Use Kafka Connect when?	Datastore code can't be changed; connector exists.
71. Two connector types?	Source and sink.
72. Who stores Connect offsets?	Workers, in internal Kafka topics.
73. RegexRouter?	Change destination topic via regex + replacement.
74. TimestampRouter?	Change topic based on message timestamp.
75. Better than JDBC polling for databases?	CDC (Debezium).
76. Local state store in Kafka Streams?	RocksDB + changelog topic.
77. Windowed join is which?	Stream–stream.
78. Single-event pattern also called?	Map/filter; stateless.
79. Multiphase processing needed when?	Result needs all data (top-10) ⇒ repartition to a single-partition topic.
80. Name two Kafka Streams examples.	Word count, stock-market statistics (also click-stream enrichment).

6 · Model-QP-only topics (outside the current syllabus — read if time permits)

The Model QP (course code 21AI72) puts **Chapter 7 "Making the data available"** in its Unit IV: 7a "elaborate any two protocols used to send data to the client", 8a "different ways to integrate filtering for each protocol", 1.7 "two drawbacks of data sync". No CIE/SEE has asked it, and the current Unit IV syllabus is Kafka only, so low probability. The Unit 4 notes do not cover this; summary below (Streaming Data Ch.7, printed pp.113–131).

Communication pattern BOOK p.114 ↗	Idea	Drawbacks
Data Sync BOOK p.115 ↗	Client gets full data set once, then only changes	Large data set / bandwidth; may not fit device; version conflicts; need merge policy Model QP 1.7
RMI/RPC BOOK p.116 ↗	API calls a method on the client when data arrives	Failure detection hard; may overwhelm client
Simple messaging BOOK p.117 ↗	Client asks for latest data, API replies	Chatty; no new-data alert; large payload after being offline
Publish-subscribe BOOK p.118 ↗	Client subscribes to a channel; API pushes on change	More complex protocol; API tracks clients

Table 7.1 — protocols [BOOK p.129 ↗](#)

Protocol	Message frequency	Direction	Latency	Efficiency	Fault tolerance
Webhooks BOOK p.119 ↗	Low	Uni (server→client, HTTP POST to registered callback)	Average	Low	None
HTTP long polling BOOK p.121 ↗	Average	Bi	Average	Average	None
Server-sent events BOOK p.123 ↗	High	Uni; single persistent HTTP connection; push-proxy mode saves mobile power	Low	High	None by default, partial
WebSockets BOOK p.126 ↗	High	Bi (HTTP handshake then TCP, full duplex)	Low	High	None by default, can be completely implemented

Filtering [BOOK p.129 ↗](#): **where** — analysis tier (aggregations), streaming API tier (raw streams), client; **type** — **static** (decided in advance, like an SQL view) vs **dynamic** (client-driven at run time, like an arbitrary query). Per protocol (Table 7.2 [BOOK p.131 ↗](#)): webhooks — capture the query at endpoint registration; long polling — encode filter in the URL; SSE — filter in the EventSource URL/query parameters; WebSockets — most flexible, client sends a message with the filter after connecting.